

Assembler, LED, and Text-Based Virtual Pet

Tangina Kader, Amy Showell, Shriya Sistla, Yao Tong

June 15, 2025

1 Assembler Implementation

Our assembler is composed of three main components: the emulate file, where instruction fetch and encode is performed; the modules, which consist of all instruction-encoding functions; and the data structures. The first data structure is the dynamic **symtable**, a dictionary of labels to addresses. The second data structure is the alias table, a dictionary of mnemonics to function pointers where the instruction type is handled.

This assembler utilizes two-pass assembly. The function **first_pass** loops through each line of the input assembly file, storing labels and their corresponding addresses in a symbol table. Next, the function **second_pass** accesses each line of the assembly file again, tokenising each line to identify the instruction name, replace labels with offsets, and extract instruction parameters. The instruction name is then inputted to the alias table's lookup function, which returns a function pointer to the proper encode function. For example, inputting the **add** instruction to lookup table returns a pointer to function **arith** in module **data-processing**. Each of these functions takes the list of tokens, the size of the token list, and an integer pointer to where the encoded instruction is to be stored. The integer pointer's contents are updated based on the instruction type and tokens. Each function also returns a status code indicating encoding success, either **EXIT_SUCCESS** or **EXIT_FAILURE**. Lastly, the encoded instruction is converted to binary and written to the specified output file. Note that in the second pass, any line consisting of only a label, directive, or newline is automatically skipped.

While writing our assembler, we learned to appreciate the importance of fleshing out useful data structures before implementing other functionality. For example, the dynamic symbol table was originally fixed-length, which is inefficient given there could be any number of labels in an assembly file. Also, the table initially stored addresses as integer pointers instead of just integers, which caused issues when looking up addresses using labels. Thus, thorough brainstorming, testing, and review are always necessary before a data structure is deployed. In addition, our group focused on the challenging implementation task we identified last week: writing utility functions early on. Throughout the assembler task, any function that seemed usable in multiple modules – like extracting immediate values and register numbers – were written in a file called **assembly-utils.c**. As a result, many of our tasks were simplified, and we enjoyed more efficient development.

2 Blinking LED Implementation

3 Text-Based Virtual Pet

A “virtual pet” is a digital toy that often requires care and attention. The most popular example is Bandai Namco’s Tamagotchi, a physical toy with a few interaction buttons and a digital screen where the “pet” is displayed. Similar to a Tamagotchi, our text-based virtual pet extension requires users to feed, clean, and play with their pet to keep the pet happy and alive. However, our implementation also includes an interaction limit to prevent users from becoming addicted or dependent on their virtual pet. Our virtual pet extension can be played as a fun game for all backgrounds, but it can also specifically be used as a fun way to help children and teens practice responsibility and balance; although users need to check on and care for their virtual pet every once in a while, they are not allowed to spend the entire day focusing only on their pet. This concept can then be generalized to any aspect of life, including studies, self-care, hobbies, and more.

3.1 Virtual Pet Design

3.2 Challenges

3.3 Testing

4 Group Reflection

5 Individual Reflections

5.1 Tangina Kader

5.2 Amy Showell

5.3 Shriya Sistla

5.4 Yao Tong